

scalable HR system using **TypeScript OOP principles**. The system will manage users, employees, managers, and departments. You'll simulate real business logic including authentication, promotion workflows, team hierarchies, salary validation, and generating reports — using classes, inheritance, access modifiers, getters/setters, abstract classes, interfaces, static methods, and enums. By the end, you'll have a system that models real-world HR operations using clean, typed TypeScript code.

Classes to Build:

✓ **User** (abstract class)

- readonly id: number
- name: string
- email: string
- private password: string
- createdAt: Date
- Method: abstract getRole(): string
- Method: authenticate(email: string, password: string): boolean

✓ **Employee** (extends **User**)

- private _salary: number
- protected department: Department
- Method: getNetSalary(): number
- Method: promote(percentage: number)
- Get/Set for salary (must be ≥ 3000)
- Overrides getRole() to return "Employee"

✓ **Manager** (extends **Employee**)

- team: Employee[] = []
- Method: addEmployeeToTeam(emp: Employee): void
- Method: removeEmployeeFromTeam(empId: number): void
- Method: getTeamReport(): string[] $\rightarrow$ array of team member info
- Overrides getRole() to return "Manager"

✓ **HR (Utility Class – no instances)**

- Static Method: generateUserId(): number
- Static Method: isValidEmail(email: string): boolean
- Static Method: calculateTax(salary: number): number → 10% tax
- Static Method: generateReport(users: User[]): string[] → printable list of users by role

✓ **Department (class)**

- name: string
- employees: Employee[]
- Method: addEmployee(emp: Employee)
- Method: getDepartmentSize()

Enum for Roles

Interface for Loginable